

TrustNode Edge Security & Architecture Whitepaper

Built for industrial operators. Read-only on the plant floor,
outbound-only on the network, and tenant-isolated end-to-end.

Audience: Plant Managers · IT Security · OT Engineering · Compliance

Document version: 2026-05-15 (rev. 2)

Maintained by the TrustNode engineering team

TrustNode Edge — Security & Architecture Whitepaper

Document version: 2026-05-15 (rev. 2) **Audience:** Plant Managers, IT Security, OT Engineering, Compliance **Software:** TrustNode Edge — industrial PLC → database gateway with optional cloud portal

A note on language

Industrial software is read by very different people — a plant manager who cares about uptime, an IT security lead who cares about firewalls and keys, an OT engineer who cares about protocols and PLCs, and a compliance officer who maps everything to standards. We wrote this document so each of them can find their part without wading through someone else's.

When we introduce a technical term for the first time, we put a short plain-language explanation next to it in a **callout box** like this:

Plain-language box — short, jargon-free explanation of the term just used.

Standards are referenced with their number *and* a one-line description, so you do not need to look them up.

How to read this document

If you are a...	Read these sections	Time
Plant / Operations Manager	§1, §2, §10, §13 (deployment options), §19 (FAQ)	8 min
IT Security	§3, §5, §6, §8, §10, §15 (network prereqs), §16 (backup)	20 min
OT Engineering	§2, §3, §4, §7, §13 (deployment options), §15 (network prereqs)	20 min
Compliance / Auditor	§9, §10, §11, §16 (backup/redundancy)	12 min
Project Manager / Architect	§13 (deployment options), §14 (storage options), §15, §16	15 min
Glossary	§20 — every acronym in plain English	5 min

1. Trust at a glance — for managers

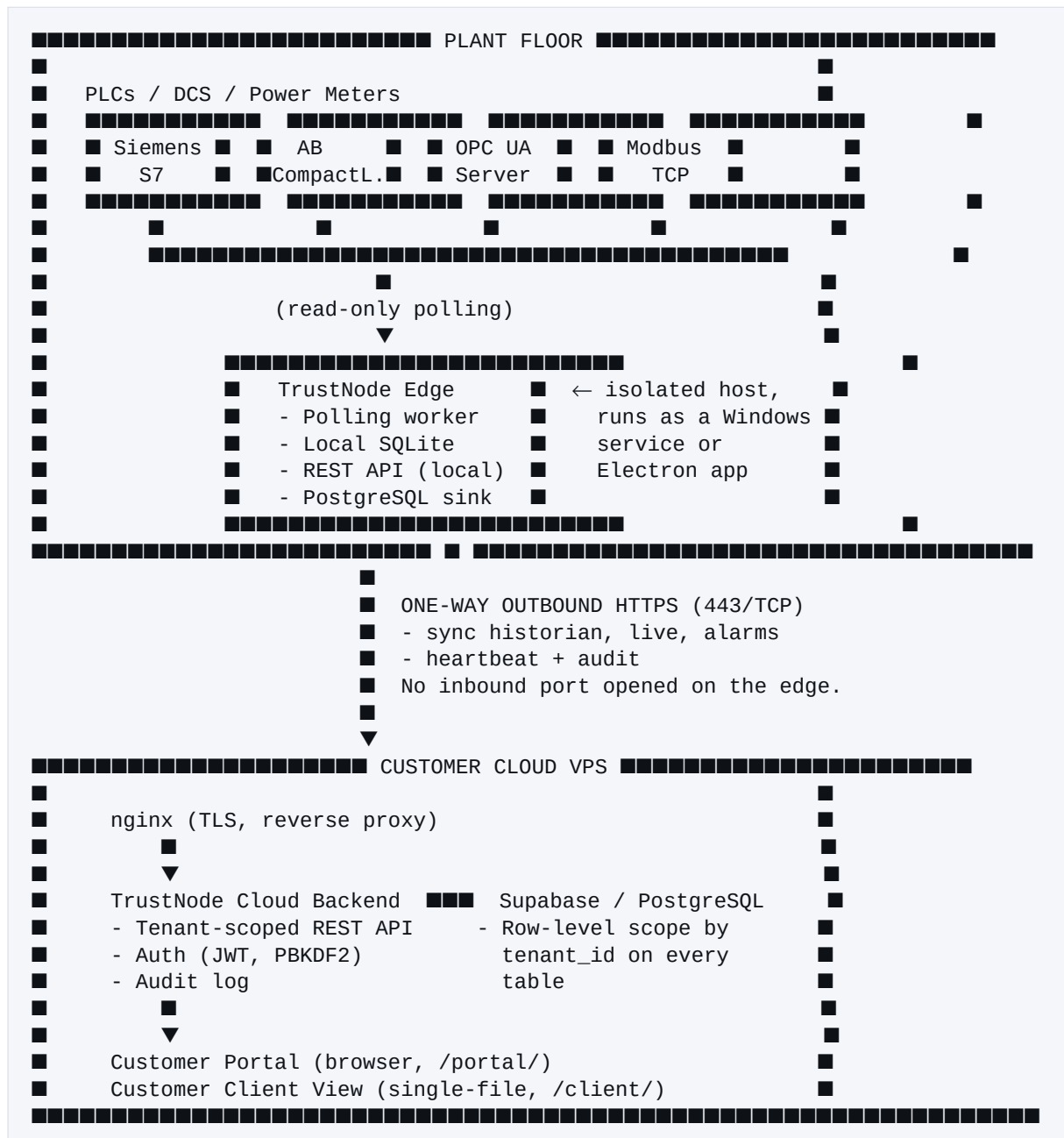
TrustNode Edge sits inside your plant network, reads from your PLCs **read-only**, and pushes that data to a database **you control**. It is built for industrial operators who cannot tolerate IT changes that risk production.

Five things to remember:

- 1. Read-only on the plant floor.** TrustNode does not write to PLCs. There is no code path in the gateway that sends commands to a PLC. Your control system is untouched.
- 2. Outbound-only network.** The edge never accepts inbound connections from the internet. It dials out to the cloud — exactly like a printer dials out to the manufacturer for firmware updates, or like a web browser asks for a webpage. Your firewall stays closed.
- 3. Your data, your database.** The customer chooses where the data lives — local-only SQLite, a PostgreSQL instance you own, or our managed Supabase. You can move it at any time.
- 4. One customer cannot see another customer.** Each customer is a *tenant*. Every database query, every API call, every live stream is filtered by `tenant_id` before it leaves the server. We re-verify this with automated checks on every release.
- 5. Designed for industry standards.** The architecture maps cleanly to the **Purdue Model** (the OT industry's reference picture of how plant networks should be layered), follows **IEC 62443** (the international standard for industrial cybersecurity), and the practices we use are aligned with **ISO 27001** (general information-security management) and **NIS2** (the EU's directive on cybersecurity for critical infrastructure).

If your CISO wants the technical evidence, §3 to §9 below are written for them.

2. What TrustNode does, in one diagram



Data flow direction: PLC → Edge → Cloud → Browser. Always that direction. The cloud cannot reach back into the edge; the browser cannot reach the PLC.

What is a PLC? A *Programmable Logic Controller* is the small industrial computer that actually controls a machine on the plant floor — opens a valve, starts a motor, weighs material. It is the most critical component on the line. We read from it. We never write.

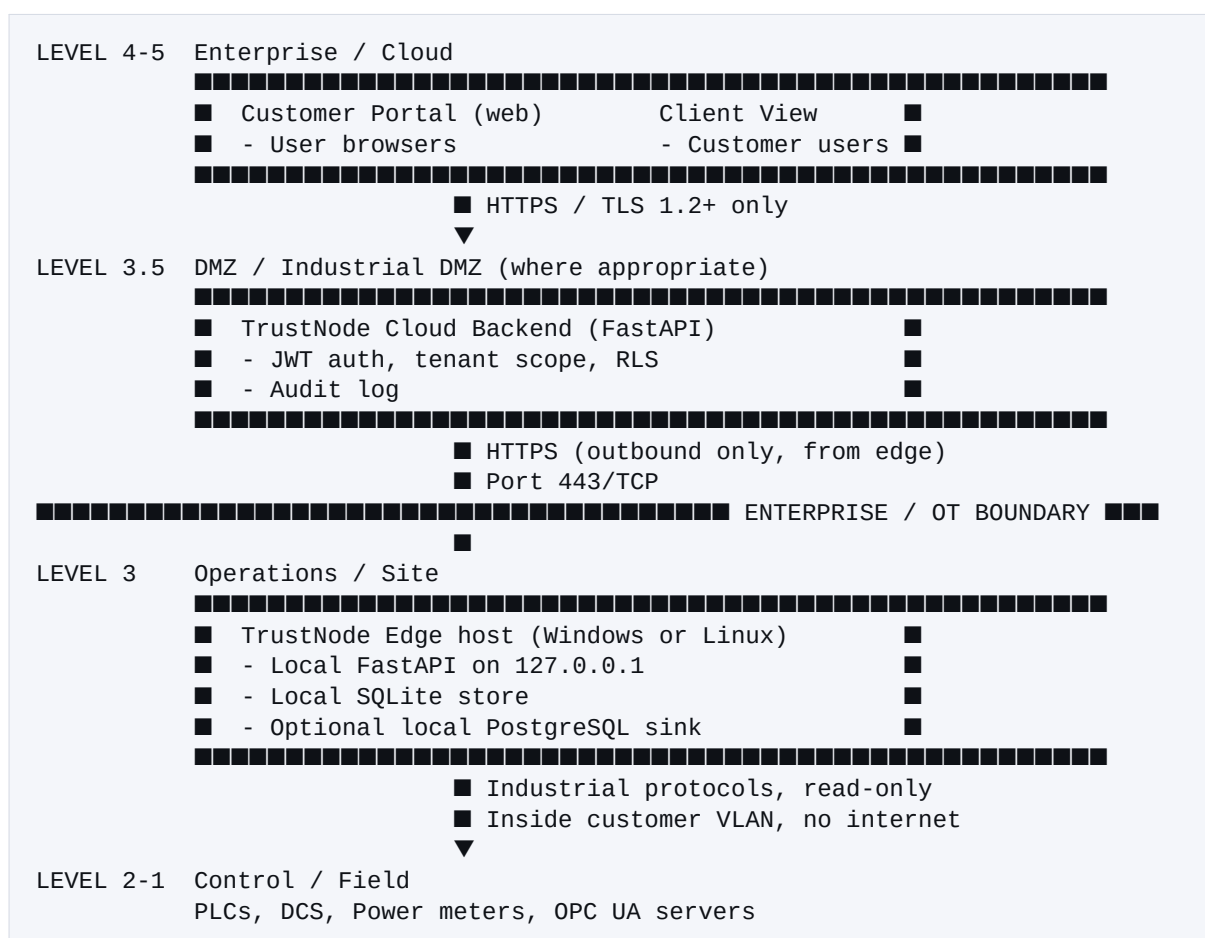
What is "the cloud"? In this document, "cloud" means a Linux server we operate (a VPS), with a database and a website, reachable at `trustnode.lapps.app` or a customer-branded domain. It is not a public cloud product like AWS S3 — it is your customer-specific server.

3. Network architecture & Purdue alignment

We map directly to the **Purdue Reference Model** for industrial control networks.

Purdue Model in one sentence. A picture of the plant network drawn as horizontal layers (Levels 0–5): the field devices at the bottom, business IT at the top. Each layer should only talk to its immediate neighbours, and traffic crossing the OT/IT boundary should be tightly controlled. The model is the industry's shared reference for "what good looks like" in plant networks.

Each level has a defined role; conduits between levels are tightly controlled.



DMZ — what does that mean? A *demilitarised zone* is a buffer network between two more sensitive networks. In industrial IT, "Level 3.5" or "Industrial DMZ" is where the proxy/gateway sits — close enough to the plant to talk to it, far enough to be isolated. Our cloud backend functions like that DMZ between your operators (Level 4–5 enterprise) and the plant (Level 0–3).

VLAN — what does that mean? A *Virtual LAN* is a way to put network devices into separate, isolated groups even though they're on the same physical switch. Plant networks usually keep PLCs on their own VLAN so office traffic cannot reach them.

What this means for IT

- **No new inbound ports on plant firewalls.** The edge initiates an outbound HTTPS session to the cloud (same direction as web browsing). No NAT translation, no port forwarding, no exposed listener.

> **NAT, port forwarding — what?** NAT (Network Address Translation) lets devices inside your network share one public address. *Port forwarding* would mean punching a hole in the firewall so the outside world could reach a specific machine. We do not need either.

- **No new inbound ports on the edge host.** The edge's REST API is bound to 127.0.0.1 (loopback). The Electron desktop talks to it locally; nothing on the LAN sees it.

> **127.0.0.1 / loopback** — a special address that means "this machine, talking to itself only." The address is not reachable from any other computer.

- **Cloud VPS exposes only TCP/443. nginx** (a common, well-audited web server) terminates TLS; the FastAPI backend listens on 127.0.0.1:8000 and is reachable only through nginx.

> **TLS terminate** — TLS is the encryption used by HTTPS. "Terminating" TLS means nginx is the only thing that holds the certificate, decrypts the request, then passes it on internally. This keeps the rest of the system simpler.

- **No reverse tunnel.** We do not use SSH reverse tunnels, ngrok, or persistent inbound WebSockets from the cloud. Cloud-side livestream uses **Server-Sent Events** — a one-way HTTP push from cloud to browser, not cloud → edge.

> **Server-Sent Events (SSE)** — a way for a server to keep an HTTP connection open and stream short updates over it. Browsers handle it natively. It only flows *server* → *browser*. The server cannot use it to reach into the browser.

What this means for OT

- The edge sits where a historian would sit. Polling cadence is configurable; default is 1 Hz per gateway, well below typical PLC scan rates.

> **Polling cadence** — how often we ask the PLC for the current value of each tag. 1 Hz means once per second. PLC scan rates are usually much faster (10–100 ms per scan).

- No PLC writes. Tag mappings define what to read; there is no corresponding "write tag" feature in the gateway code.
- Edge failure does not affect the PLCs. If the edge is powered off, the plant runs exactly as before; we only buffer data.
- Network share: only one VLAN connection (read-only polling) and one outbound NAT path (to cloud HTTPS) are needed. No multicast, no industrial broadcast, no SCADA-control traffic.

TrustNode in the Purdue Reference Model

Vendor-neutral picture of where TrustNode sits inside an industrial network.

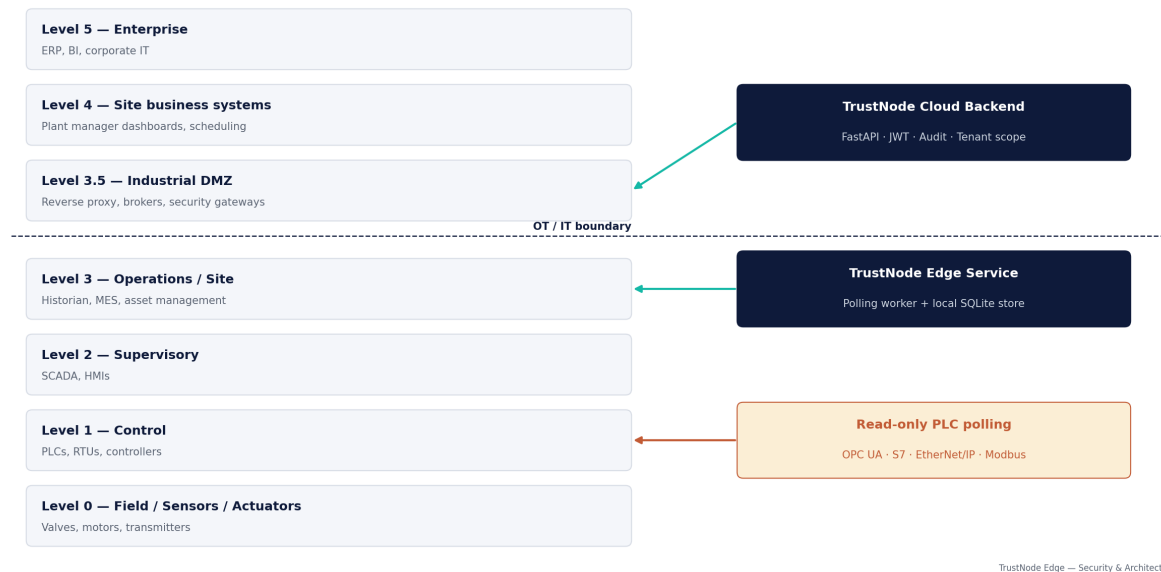


Figure 1 — Purdue model with TrustNode.

4. Industrial protocols, in plain language

The edge supports four PLC/automation protocols today, all **read-only**. Here is what each one is, why it exists, and how we treat it.

What is an "industrial protocol"? PLCs from different vendors speak different "languages" over the network. An industrial protocol is the agreed-upon way of asking the PLC "what is the current value of tag X?" without disturbing its main job. The four protocols below cover the vast majority of factory floors.

OPC UA — the modern, vendor-neutral standard

- **What it is:** *Open Platform Communications — Unified Architecture*. An open, vendor-neutral industrial protocol standardised as **IEC 62541**.
- **Used for:** Siemens (the recommended path on modern Siemens PLCs), Beckhoff, OPC UA servers in front of any PLC, modern field devices.
- **What we use it for:** Reading tag values. We connect, the server returns a value and a *quality flag* (good / uncertain / bad), we record both.
- **Security features we honour:** OPC UA itself supports user/password auth and end-to-end encryption; we pass through whatever the customer's server accepts.
- **Why it is safe in our hands:** The library we use only exposes read methods in our code paths. We do not import any write API.

Siemens S7 (Snap7) — the native Siemens protocol

- **What it is:** Siemens's own protocol for S7-300, S7-400, S7-1200 and S7-1500 PLCs. We use the open-source `python-snap7` library.
- **Used for:** Reading data blocks, memory areas (M, I, Q), bit fields. Common on legacy Siemens deployments where OPC UA is not exposed.
- **What we use it for:** Reading. Tag format is `DB_no.byte.bit` — i.e. *data block number, byte offset, bit offset*.
- **Why it is safe in our hands:** Same as above — only read calls. The PLC's own access controls (PUT/GET, password) still apply on top.

Allen-Bradley / Rockwell (EtherNet/IP, CIP) — the dominant US/UK standard

- **What it is:** Rockwell's family of PLCs (CompactLogix, ControlLogix) speaks **EtherNet/IP** (also called CIP — *Common Industrial Protocol*). We use `pycomm3` as primary and `pylogix` as a fallback for older PLCs.
- **Used for:** ControlLogix tags, structures, UDTs (user-defined types).
- **What we use it for:** Reading tag values by tag name (the symbolic name the controls engineer defined inside the PLC programme).
- **Why it is safe in our hands:** We do not bind the `write_tag()` or similar methods anywhere in the gateway code.

Modbus TCP — the universal lingua franca for field devices

- **What it is:** A simple, register-based protocol that almost every industrial device speaks. Originally serial (RS-485), now mostly over Ethernet as **Modbus TCP**. We use `pymodbus`.
- **Used for:** Power meters, energy meters, generic VFDs (variable-frequency drives), older PLCs, sensors with built-in Ethernet.
- **What we use it for:** Reading *holding registers* and *input registers* — small numeric values like voltage, current, kW, frequency, temperature.
- **Why it is safe in our hands:** Same read-only discipline.

What "read-only" actually means in code

A search of the gateway worker class returns **zero matches** for any of these write methods: `write_tag`, `write_node`, `write_*`, `set_value`, `db_write`, `write_holding`, `write_coil`. The protocols' write APIs exist in the libraries — we just don't bind them. If a future customer ever needs write capability, that becomes a deliberate, scoped, audited feature — not an accident.

5. Authentication & access control

What is authentication vs. authorisation? *Authentication* is "who are you?" — proving identity. *Authorisation* is "what are you allowed to do?" once we know who you are. The two work together but are separate decisions.

How users sign in — JWT

- **JWT** stands for *JSON Web Token* — an internet-standard format (RFC 7519) for a self-contained, cryptographically-signed identity token. After login, the browser carries a JWT on every subsequent request. The server checks the signature on every call.
- **Algorithm: HS256** — *HMAC with SHA-256*. The server signs the token with a secret key only it knows; anyone with the public token can verify it has not been tampered with, but only the server can issue new ones.

> **HMAC, SHA-256 — what?** SHA-256 is a one-way mathematical function ("hash") that turns any input into a fixed-size fingerprint. HMAC combines a hash with a secret key so that the result also proves the data has not been altered. Used in HTTPS, in OAuth, in Bitcoin, and now in our login system.

- **12-hour token lifetime** with an explicit exp (expiry) claim. Tokens past expiry are rejected at the backend, even if the browser still has them.
- **Secret source:** environment variable `TRUSTNODE_AUTH_SECRET` (rotatable), with a fallback to a per-installation persistent secret. The customer can rotate without losing data.

How passwords are stored — PBKDF2

- **PBKDF2** stands for *Password-Based Key Derivation Function 2*. An internet standard (RFC 8018) for slowing down password-cracking attempts.

> **Plain language.** When a user types their password, we run it through 120,000 rounds of cryptographic stretching before storing the result. To check the password later, we run the typed value through the same 120,000 rounds and compare. This makes guessing the password by brute force computationally expensive.

- **Iterations: 120,000.** Each login attempt costs roughly 100 milliseconds of CPU on the server. An attacker would face the same cost per guess.
- **Salt: per-user.** A random value mixed into every password before hashing so that two users with the same password get different hashes.
- **No plaintext stored, ever.** Passwords are never logged, never emailed, never written to disk in clear form.

Three roles, enforced server-side

Role	What they see	What they can change
Master / global admin (developer)	All tenants, all customers	Everything <i>except</i> edge runtime config from web
Customer admin	Only their own tenant	Their tenant: dashboards, reports, users, alarms
Customer client viewer	Only their own tenant, only modules their admin enabled	Nothing — read-only

Role enforcement is in the backend, not in the browser. The frontend respects the role for UX, but a tampered browser cannot escalate because every API call re-validates the JWT on the server.

Tenant scope check on every API call

The cloud backend runs every authenticated request through `_scoped_tenant()` before it touches the database. The function:

1. Reads the caller's JWT `tenant_id` claim.
2. Compares it to the tenant ID the request is targeting (URL or header).
3. Returns **403 Cross-tenant access denied** unless they match — *or* the caller is the global admin.
4. For write actions, additionally requires `role=admin`.

This is the single chokepoint we rely on; we keep it small and audit it. Sub-resources that are addressed by their own identifier (e.g. license modules) additionally resolve the parent record's owning tenant and re-apply the same check.

What does "403" mean? A standard HTTP response code meaning *Forbidden*. Different from 401 (*Unauthenticated*). 401 says "we don't know who you are"; 403 says "we know who you are and you can't do this."

Three roles, enforced server-side

Same login endpoint, three very different views — all gated by the JWT.

Master / Global Admin TrustNode developer team	Customer Admin Customer's IT/operations lead	Customer Client Viewer Customer staff / contractor
Can do: ✓ See all tenants & customers ✓ Manage tenants & licenses ✓ Audit log read access ✓ Issue activation codes for any tenant Cannot do: ✗ Edge runtime config from web	Can do: ✓ See only own tenant ✓ Create dashboards & reports ✓ Manage their own users ✓ Issue activation codes for own tenant Cannot do: ✗ Touch other tenants ✗ Change licenses or modules	Can do: ✓ Read-only ✓ Only modules admin enabled ✓ Only own tenant's data ✓ No admin tools Cannot do: ✗ Any write action ✗ See other users in tenant

TrustNode Edge — Security & Architecture

Figure 2 — Three roles enforced server-side.

6. Data protection

What data exists, and where

Data class	Location	Retention	Encryption
Plant historian (tag readings)	Local SQLite at the edge + optional PostgreSQL/Supabase mirror	Customer-controlled retention policy	TLS in transit; storage encryption depends on customer's DB provider (most cloud DBs encrypt at rest)
Live snapshot (latest tag values)	live_latest table at edge + cloud mirror	Tenant-keyed primary key, latest value only	TLS in transit
Configuration (gateways, tags, dashboards, reports)	Edge SQLite, cloud Supabase mirror	Versioned, soft-deleted	TLS in transit
User credentials	Edge SQLite, cloud Supabase	PBKDF2 hash; salt per user; no plaintext	Hash-only, never decrypted
JWT tokens	Client-side (localStorage per host)	12 hours	HS256 signed; bearer in Authorization header
Audit log	cp_security_audit_log table	Append-only	Per-tenant index; rows immutable

In-transit encryption

- **TLS** stands for *Transport Layer Security* — the modern name for what used to be called SSL. It's the lock icon in your browser. We use TLS 1.2 or higher everywhere.
 - **Browser ↔ Cloud:** HTTPS only on hosted deployments. Our nginx vhost forces TLS; mixed-content is impossible because the frontend computes the WebSocket scheme from `window.location.protocol` and upgrades `http → ws`, `https → wss` automatically.
- > **Mixed content** — when a secure page accidentally loads or talks to an insecure resource. Browsers block this. We make it structurally impossible.
- **Edge ↔ Cloud:** HTTPS to `https://trustnode.lsassps.app` (or your custom domain). Certificate pinning can be added via deployment config if your CISO requires it.
- > **Certificate pinning** — locking the edge to a specific TLS certificate so that even a stolen, valid-looking certificate from another authority cannot impersonate the cloud.
- **Live data stream:** Server-Sent Events (`text/event-stream`) — one-way push from cloud to browser, authenticated per-connection by JWT.
 - **PostgreSQL connections:** support `sslmode=require` via the standard psycopg driver; we recommend enabling it on every customer deployment.

At-rest encryption

We deliberately do **not** layer application-level encryption on top of the database for these reasons:

1. Industrial customers want their data queryable from the same database with their existing BI tools (Power BI, Tableau, Grafana). Application-layer encryption breaks that.
2. The cloud DB providers we recommend (Supabase managed Postgres, AWS RDS, Azure PostgreSQL) all encrypt at rest with customer-managed keys.
3. Where stronger isolation is needed, the deployment uses **separate physical PostgreSQL instances per customer** rather than a shared one.

For the edge SQLite file, customers who require disk-level encryption can place the data directory on a **BitLocker** (Windows), **LUKS** (Linux), or **FileVault** (Mac) volume. The path is configurable.

What's the difference between "at rest" and "in transit"? *In transit* = data moving across a network. *At rest* = data sitting on a disk. They are protected by different mechanisms (TLS for transit; disk/database encryption for rest).

What we never persist

- Plaintext passwords (we store PBKDF2 hashes only).
- Live tokens past their 12-hour expiry.
- Activation codes (we store SHA-256 hashes; the plaintext is shown once to the admin and expires in 30 minutes).
- Password reset tokens (hash only; 15-minute TTL; one-shot).

TTL — what does that mean? *Time to live*. How long a token stays valid before it expires automatically. A short TTL limits the damage of a leaked token.

7. Edge hardening

Local API isolation

The edge backend binds to 127.0.0.1 only. Nothing on the customer LAN — no laptop, no PLC, no neighbouring server — can hit the edge's REST API. The Electron desktop UI talks to it over loopback.

Service identity

The edge runs as a dedicated Windows Service (or Linux systemd unit). The service account has read access to its config directory and read/write to its data directory — nothing else. It is not the SYSTEM account; it cannot install software, change system settings, or read other users' data.

No remote desktop, no remote shell

The edge does not run an SSH server, RDP listener, or remote shell. The only outbound traffic it makes is HTTPS to the cloud backend. If the customer needs to reconfigure something remotely, the customer's admin signs into the cloud portal — the cloud backend then syncs the new config down to

the edge through the same outbound channel.

Why does this matter? Two of the biggest industrial breach categories in the last decade were remote-desktop tools left exposed and SSH servers without strong keys. We simply do not run either.

Read-only at the protocol layer

We do not import write methods from any of the industrial libraries in our gateway worker. The code that talks to PLCs only calls `read_*` / `get_*` methods. A future audit can confirm this with a simple `grep`.

Store-and-forward resilience

If the cloud connection drops, the edge continues polling PLCs and recording into its local historian. When connectivity returns, it back-fills the cloud incrementally. This protects against:

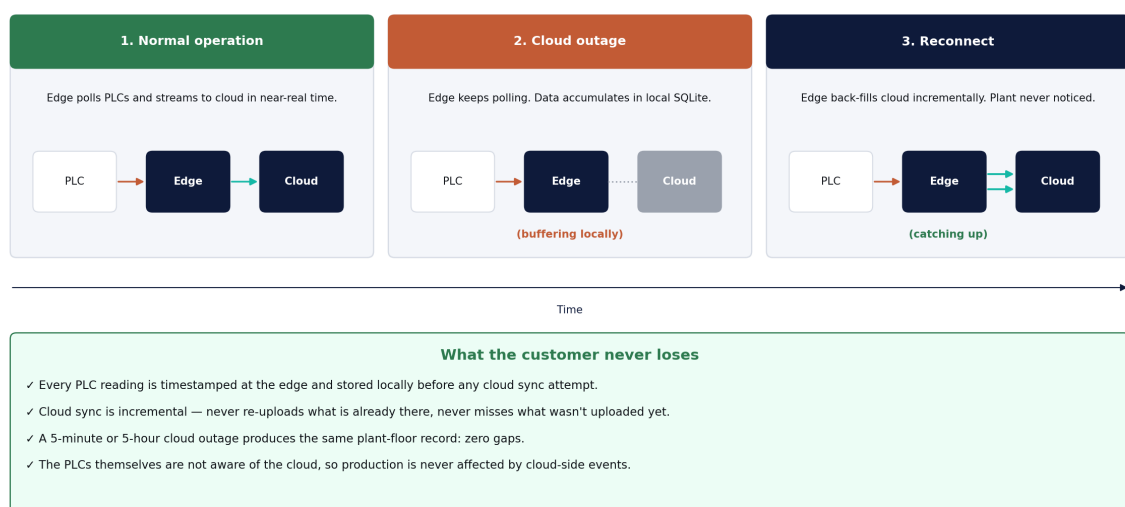
- Internet outages
- Cloud provider maintenance
- Cloud-side rate limiting

The plant floor never knows; PLC reads continue at their configured cadence.

Why is store-and-forward important? Plants run 24/7. If a 5-minute internet outage caused 5 minutes of data loss, your shift report would be incorrect. Store-and-forward means a 5-minute outage produces zero data loss — the edge holds the data and catches up automatically.

Store-and-forward resilience

What happens during a cloud outage — and what happens when it recovers.



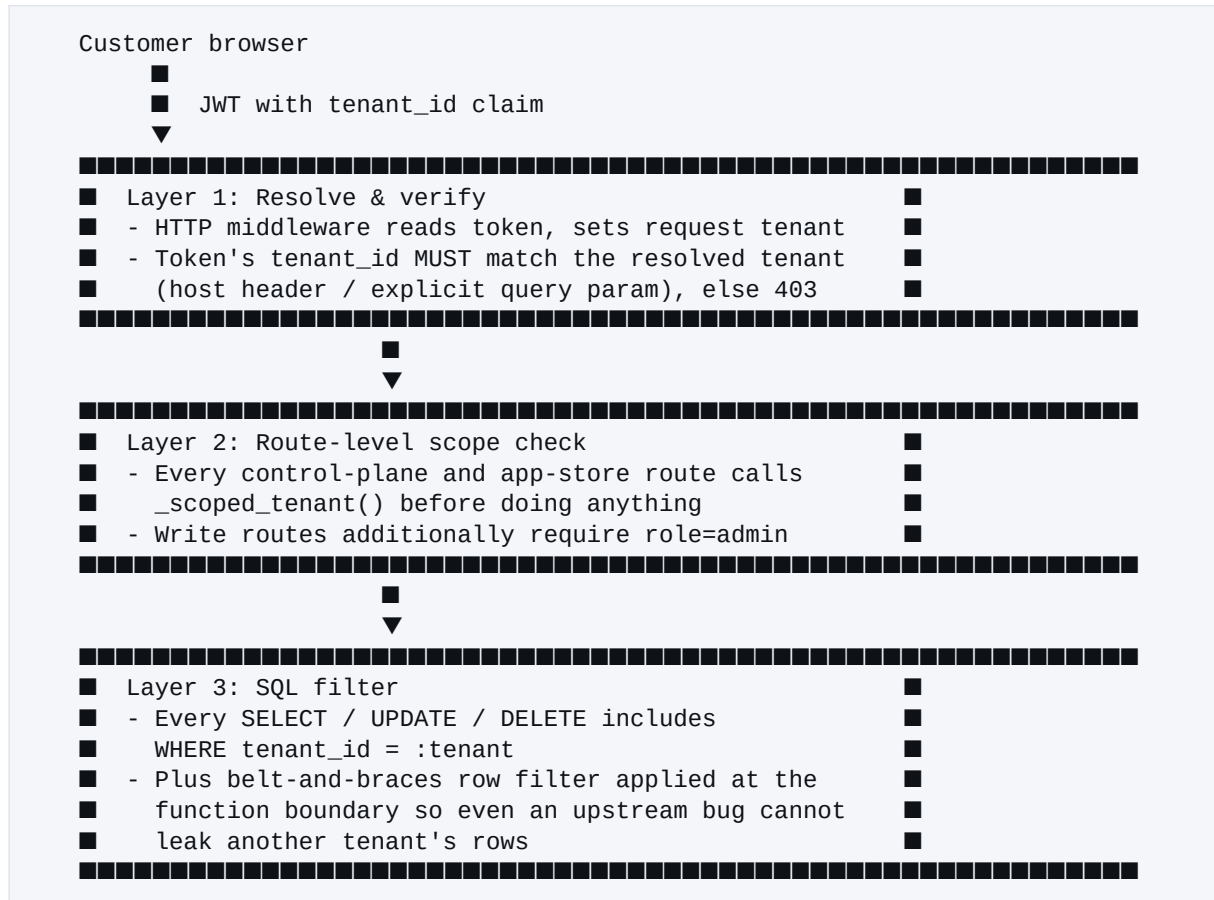
TrustNode Edge — Security & Architecture

Figure 3 — Store-and-forward resilience.

8. Cloud isolation — how tenants stay separate

What is a "tenant"? In multi-customer software, each customer organisation is a *tenant*. The architecture's job is to make sure tenants never see each other. This is the most critical promise we make to enterprise customers.

Three lines of defense



Per-tenant row-level scope (RLS-equivalent)

RLS — what is it? *Row-Level Security*. A database feature (built into PostgreSQL) that lets the database itself reject a query if it touches rows that don't belong to the current user. Even if the application code is buggy, the database refuses to return the wrong data. We use a *tenant_id* filter pattern that is mathematically equivalent to RLS, and on Supabase customers can additionally enable the database-enforced version.

Every table that holds customer data has a *tenant_id* column. The application contract is: *no query touches such a table without including* `WHERE tenant_id = :tenant`. We enforce this with:

- A single helper (`_current_tenant_id()`) that the data layer pulls from a per-request context variable.

- SQL parameters bound at query build time — there is no string interpolation of `tenant_id`. (This also blocks SQL injection on the tenant value.)
- For deployments on Supabase, customers may additionally enable PostgreSQL **Row-Level Security policies** keyed on `tenant_id`, providing a database-enforced backstop even if a misbehaving service tried to bypass the application filter.

Continuous verification

We ship three automated smoke scripts that verify segregation against the live deployment:

Script	What it tests	Checks
smoke-portal-end-to-end.ps1	Full provision: customer → license → edge → activation → users	7 steps
smoke-tenant-segregation.ps1	Cross-tenant read/write blocked across customers, edges, licenses, users, activation codes	17 checks
smoke-portal-spa-flows.ps1	The exact API calls the portal SPA makes for master/customer admin/client viewer	25 checks

These run against the production VPS and report pass/fail per check. We re-run them on every release. As of this revision, the live production system passes **49/49** segregation checks.

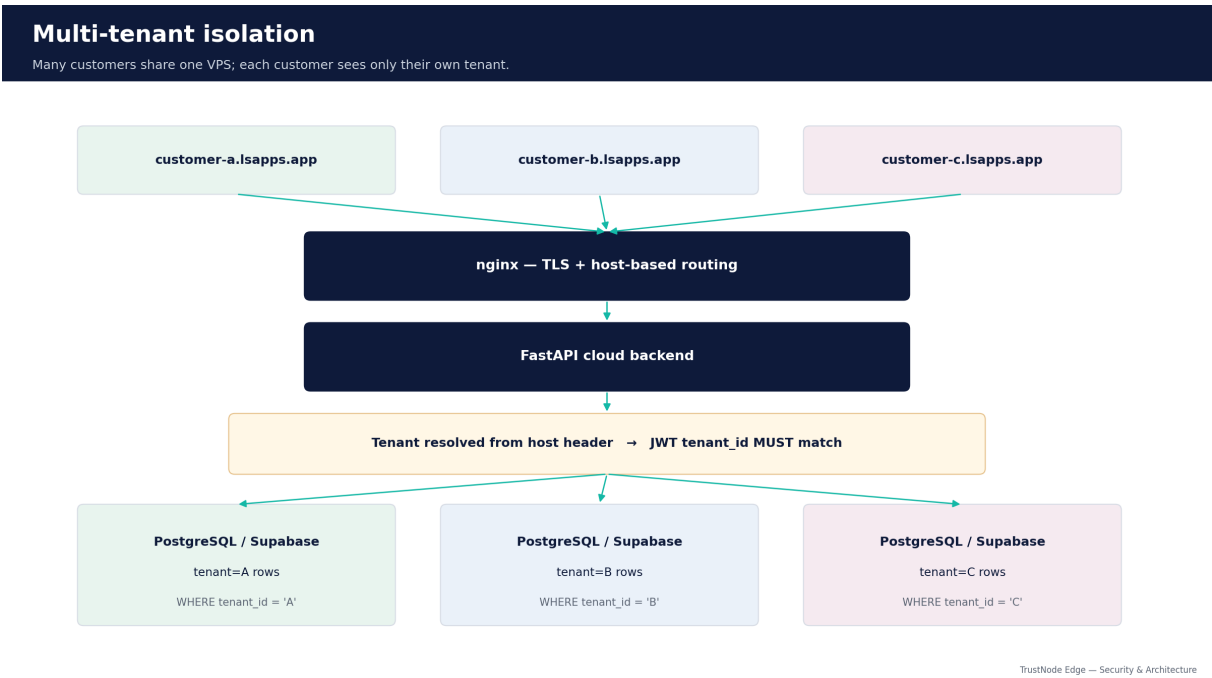


Figure 4 — Multi-tenant isolation.

9. Audit trail

Every administrative action against the control plane writes one row to `cp_security_audit_log`:

Column	What it records
<code>id</code>	Monotonic primary key
<code>ts_utc</code>	UTC timestamp
<code>actor_type</code>	user, device, or system
<code>actor_id</code>	JWT subject or edge ID
<code>tenant_id</code>	The tenant the action affected
<code>action</code>	e.g. <code>user.upsert</code> , <code>license.modules.set</code> , <code>edge_link.unlink</code>
<code>outcome</code>	<code>ok</code> , <code>error</code> , <code>not_found</code>
<code>correlation_id</code>	Inbound X-Correlation-Id for tracing
<code>details_json</code>	Compact JSON with context (e.g. <code>{"username": "alice"}</code>)

The table is **append-only** in practice — no application code deletes rows. An index on (`tenant_id`, `ts_utc` DESC) keeps queries fast even after years of data.

Why does an audit log matter to a manager? Two reasons. First, regulatory compliance (NIS2, ISO 27001) requires it. Second, when something looks wrong — "who deleted that gateway last week?" — the audit log gives you a precise, time-stamped answer instead of guesses.

Representative actions audited today:

- `customer.upsert`, `customer.delete`
- `edge.upsert`, `edge.delete`, `edge.heartbeat`
- `license.upsert`, `license.delete`, `license.modules.set`
- `user.upsert`, `user.delete`
- `activation_code.issue`, `activation_code.apply`, `activation_code.update`, `activation_code.delete`
- `password_reset.issue`, `password_reset.apply`
- `tenant.upsert`
- `edge_link.unlink`

For SOC 2 / ISO 27001 audits, this log answers "who did what, when, scoped to which tenant, with what outcome."

10. Compliance mapping — what each standard is, and how we meet it

This is not a certification claim — TrustNode is not currently certified to these standards. The table maps our existing engineering practices to the controls each standard expects so your compliance team can evaluate fit.

The four standards we map to — in plain language

- **IEC 62443** — *Security for Industrial Automation and Control Systems*. The international standard specifically written for OT cybersecurity. Published by the IEC (the International Electrotechnical Commission) jointly with ISA (the International Society of Automation). It's the most relevant standard for what we do.
- **NIS2** — *Network and Information Systems Directive 2*. An EU directive (in force since October 2024) that requires operators of essential services (energy, water, transport, manufacturing, healthcare, digital infrastructure) to manage cybersecurity risks and report incidents. If you operate in the EU and are in a regulated sector, NIS2 applies to you.
- **ISO/IEC 27001** — *Information Security Management Systems*. The general international information-security standard, applicable to any organisation. Defines a set of controls (Annex A) covering everything from policies to physical security. Used worldwide by IT and infosec functions.
- **Purdue Reference Model** (ANSI/ISA-95) — Not a security standard, but the universally-used reference picture of an industrial network's layers. Auditors will ask "where on the Purdue model do you sit?" because that tells them what risks apply.

How we map

Control area	Standard	What TrustNode does today
Zone & conduit	IEC 62443-3-2	Edge sits in OT zone; cloud sits in Enterprise/Cloud zone; the only conduit is outbound HTTPS over port 443.
Restricted data flow	IEC 62443-3-3 SR 5.x	Edge → cloud is the only conduit. No inbound to edge, no PLC writes from cloud.
User identification & auth	IEC 62443-3-3 SR 1.1; ISO 27001 A.5.16	JWT auth with PBKDF2 password hashing; per-tenant role-based access.
Use control & least privilege	IEC 62443-3-3 SR 2.x; ISO 27001 A.5.18	Three roles: master / customer-admin / client. Customer-admin cannot touch other tenants.
System integrity	IEC 62443-3-3 SR 3.x	TLS for transport; PBKDF2 + JWT signatures; immutable audit log.
Data confidentiality	IEC 62443-3-3 SR 4.x; ISO 27001 A.8.24	TLS in transit; at-rest encryption available via DB provider and/or volume encryption.
Resource availability	IEC 62443-3-3 SR 7.x	Edge store-and-forward keeps polling during cloud outages.
Timely response to events	IEC 62443-3-3 SR 6.x; ISO 27001 A.5.25	Append-only audit log with per-tenant index.
Information security policies	ISO 27001 A.5.1	Public security whitepaper (this document).
Access control	ISO 27001 A.8.2, A.8.3	JWT + tenant scope at every API call; deny-by-default.
Cryptography	ISO 27001 A.8.24	HS256 JWT, PBKDF2-HMAC-SHA256 (120k iter), SHA-256 token hashes, TLS 1.2+.
Logging & monitoring	ISO 27001 A.8.15	Per-action audit log with correlation IDs.

Control area	Standard	What TrustNode does today
Supply chain (software)	ISO 27001 A.5.20–A.5.23	Pinned Python deps; lockfile-based JS deps; Electron kept current.
NIS2 — risk management measures	NIS2 Art. 21(2)(a)	Documented architecture (this doc); patched dependencies; segregated tenants.
NIS2 — incident handling	NIS2 Art. 21(2)(b)	Audit log enables incident reconstruction; one-shot activation codes prevent replay.
NIS2 — business continuity	NIS2 Art. 21(2)(c)	Edge store-and-forward; customer-chosen DB provider with provider-level backups.
NIS2 — supply-chain security	NIS2 Art. 21(2)(d)	Pinned versions; reproducible CI builds; this document for downstream review.
Purdue Model L0–L5	ISA-95 / Purdue Reference Model	Edge at L3 (Operations); cloud at L4–L5; no traffic crosses without TLS + auth.

11. Continuous hardening

Security is a process, not a property. Concretely:

- **Smoke tests on every release.** Three smoke scripts run against the production VPS. A failure blocks the release.
- **Dependencies pinned + reviewed.** Backend Python is pinned to patch level; frontend JS uses lockfiles; Electron is kept on the current major.
- **Single chokepoint for tenant scope.** All cross-tenant decisions flow through one function (`_scoped_tenant`). Changes to it require code review.
- **Audit log is append-only.** No code path deletes from `cp_security_audit_log`.
- **One-shot tokens for sensitive transitions.** Activation codes (30 min TTL) and password reset tokens (15 min TTL) are single-use and stored as hashes only.
- **Outbound-only edge.** Even if your IT team finds an issue, the blast radius is contained — the edge has no inbound listener.

12. What we ask of you, the customer

To get the security posture this document describes, the deployment must do these five things. Most are defaults; we call them out so you can confirm.

1. **TLS everywhere.** Use HTTPS for any browser-facing deployment. The cloud VPS template ships with this.
2. **Database with SSL.** If you bring your own Postgres, set `sslmode=require` in the connection string we configure.
3. **Edge in OT VLAN, no inbound.** The host running the edge service should be reachable from PLCs, but the only outbound traffic it needs is HTTPS/443 to the cloud.
4. **Disk encryption on the edge host.** BitLocker / LUKS / FileVault at the OS level protects the SQLite file even if the host is stolen.
5. **Strong password policy for portal users.** PBKDF2 with 120k iterations makes brute-force expensive, but it's not magic.

If your environment has additional requirements (FIPS-validated TLS, certificate pinning, SAML SSO, mTLS between edge and cloud, customer-managed encryption keys), reach out — these are deployment options we support on request.

SAML SSO, mTLS — what? *SAML SSO* lets your employees sign in with your corporate identity (e.g. Azure AD, Okta) rather than a TrustNode-specific password. *mTLS* (mutual TLS) means both sides of a connection prove their identity with a certificate, not just the server. Both are common requirements for high-security industrial deployments.

13. Deployment topologies — five ways to install TrustNode

TrustNode is built to fit different customer realities. The software is the same; only the box it runs on and the network around it changes. Five common shapes are documented below, with a diagram, a "best for", and the implications for IT/OT each one carries.

What is a "topology"? The shape of the deployment — *where the software runs and what it can reach*. A topology answers questions like "is there a cloud?", "is the edge in the panel or in the server room?", "is the data on-prem or hosted?". Picking the right topology is the first big decision in every project.

A. Plant PC / desktop install

See **Figure A** (`deployment_plant_pc.png`).

- **What it is:** TrustNode runs on a regular Windows 10/11 PC that already sits inside the plant — often a spare HMI workstation or a small office tower.

- **Best for:** small to medium plants, single-site operations, customers who explicitly do not want any cloud component, pilots and proof-of-concepts.
- **What you need:** the PC needs an Ethernet route to the PLCs (often a second NIC into the OT VLAN), Windows 10/11, and roughly 16 GB RAM + 256 GB SSD. We install the TrustNode service; it auto-starts.
- **Data location:** local SQLite file at `C:\TrustNode\data\` by default. Customer keeps everything.
- **What it does NOT need:** any internet access. The plant PC works offline.

B. Industrial PC (IPC) in the electrical panel

See **Figure B** (`deployment_ipc_panel.png`).

- **What it is:** a rugged, fanless DIN-rail PC mounted inside the electrical cabinet next to the PLCs. Designed for vibration, dust, 24 VDC power.
- **Best for:** machine builders shipping a turnkey line, OEMs who want TrustNode to be part of the deliverable, customers who don't have a server room.
- **What you need:** an IPC with ≥ 8 GB RAM, an SSD or industrial SD card, a 24 VDC supply. Common models: Siemens IPC127E, B&R APC910, Beckhoff CX-series, generic Advantech UNO. We pre-load Windows or Linux and the TrustNode service.
- **Data location:** local SSD/SD by default. Customer keeps everything.
- **Optional add-on:** a cellular modem or VPN router fitted in the same panel can give the IPC outbound internet for a cloud portal. The IPC still works fully without one.

C. Customer server in their datacenter

See **Figure C** (`deployment_customer_server.png`).

- **What it is:** TrustNode runs on a Linux VM (Ubuntu/RHEL/Debian) or a rack server in the customer's existing datacenter. Often this single server polls several plants over the customer's WAN.
- **Best for:** mid-to-large industrial customers with an internal IT team and a datacenter, customers who already host their own PostgreSQL or SQL Server, customers who must keep all data on-prem.
- **What you need:** a VM with 4 vCPU / 8 GB RAM (start) and routable network access to the PLCs at each plant (over the customer's MPLS, SD-WAN, or VPN). We hand over a Docker image or a tarball + systemd unit.
- **Data location:** the customer's own PostgreSQL (or other DB — see §14). Backups, replication, and HA come from the customer's existing IT practices.

D. Cloud-bridged (the reference topology)

See **Figure D** (`deployment_cloud_bridged.png`).

- **What it is:** an edge service at each plant (PC, IPC, or VM) talks outbound to a cloud backend hosted either on our managed VPS or on the customer's own VPS. Operators sign in to a web portal.

- **Best for:** customers who want web access from anywhere, multi-site rollouts, projects where the customer wants us to host.
- **What you need at the plant:** any of A / B / C above. **What you need in the cloud:** a Linux VPS (4 vCPU / 8 GB / 50 GB SSD is plenty for most fleets) with TLS, nginx, FastAPI, and PostgreSQL (or Supabase as a managed alternative).
- **Data location:** cloud Postgres/Supabase, plus a buffer at each edge for store-and-forward.

E. Multi-plant central historian

See **Figure E** (deployment_multi_plant.png).

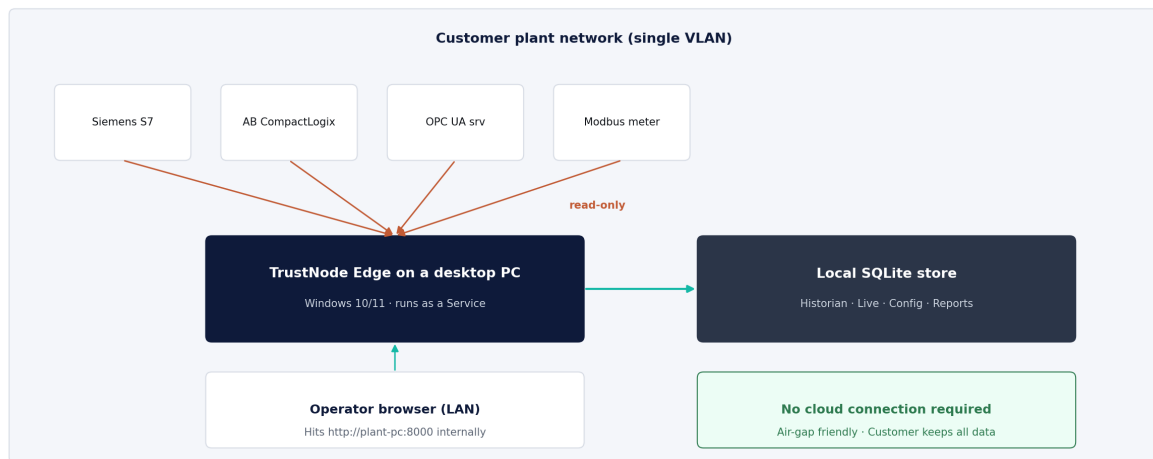
- **What it is:** several plants of the same customer pour their data into a single central historian. Operators see all plants in one portal, filtered by plant.
- **Best for:** customers with 3+ plants, multinational manufacturers wanting one source of truth, energy / utility companies aggregating sites.
- **What you need:** one edge per plant (A / B / C), one central server (C or D), one shared database. We set `tenant_id = customer_id` and use `edge_id` to filter per plant.
- **Data location:** central PostgreSQL (customer datacenter or cloud). Each plant's data is tagged with its own `edge_id`.

Picking a topology — quick guide

Customer profile	Recommended topology
One small plant, no IT team	A — Plant PC
Machine builder / OEM shipping with the line	B — IPC in panel
Mid/large customer with their own DC	C — Customer server
Wants web access from anywhere	D — Cloud-bridged
Multi-plant operator wanting one view	E — Multi-plant central
Strict air-gap, no cloud allowed	A or B (no cellular)

Deployment A — Plant PC / desktop install

Simplest setup: TrustNode runs on a Windows PC inside the plant. No cloud required.

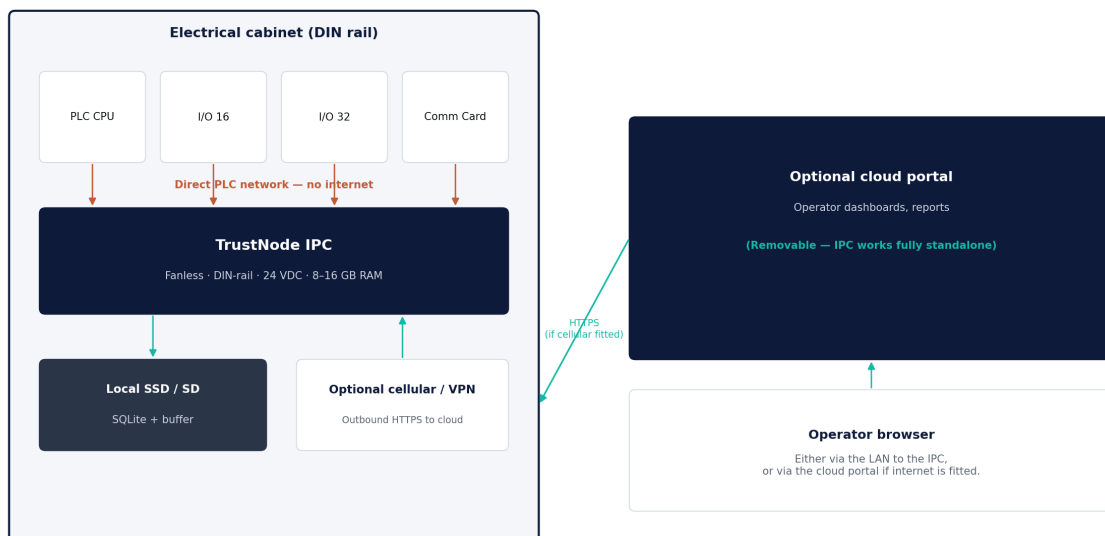


TrustNode Edge — Security & Architecture

Figure A — Plant PC / desktop install.

Deployment B — Industrial PC (IPC) in the electrical panel

Rugged DIN-rail PC mounted inside the cabinet, next to the PLCs.

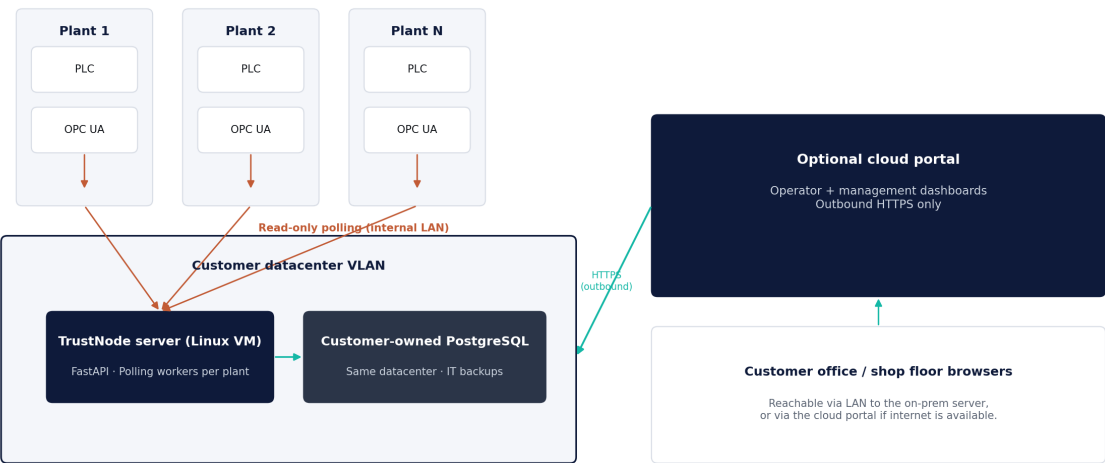


TrustNode Edge — Security & Architecture

Figure B — Industrial PC (IPC) in the electrical panel.

Deployment C — Customer server in their datacenter

TrustNode on a Linux VM or rack server. Often serves several plants from one box.

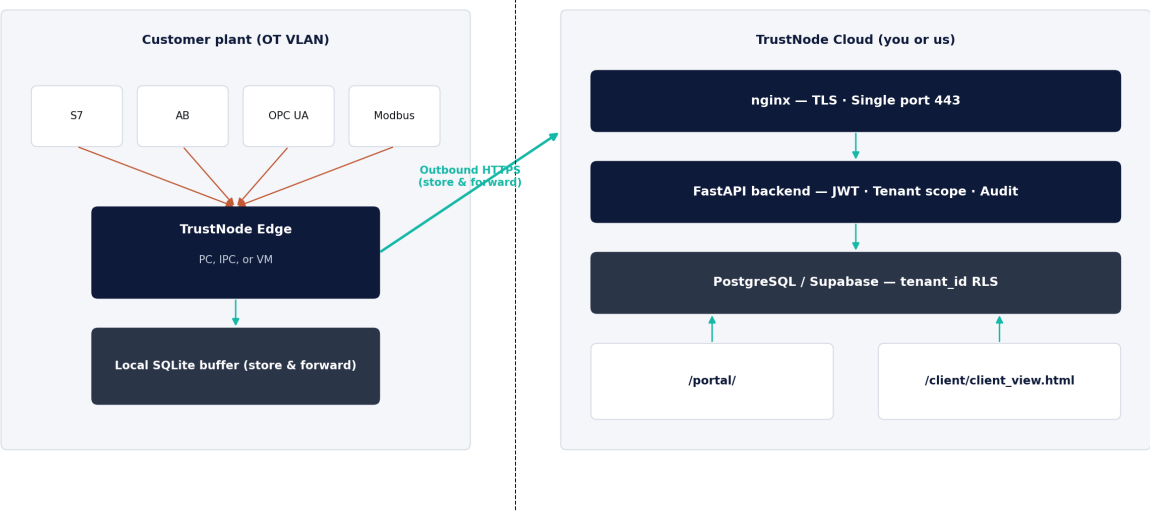


TrustNode Edge — Security & Architecture

Figure C — Customer server in their datacenter.

Deployment D — Cloud-bridged (default)

Edge in the plant + portal in the cloud. The reference topology.

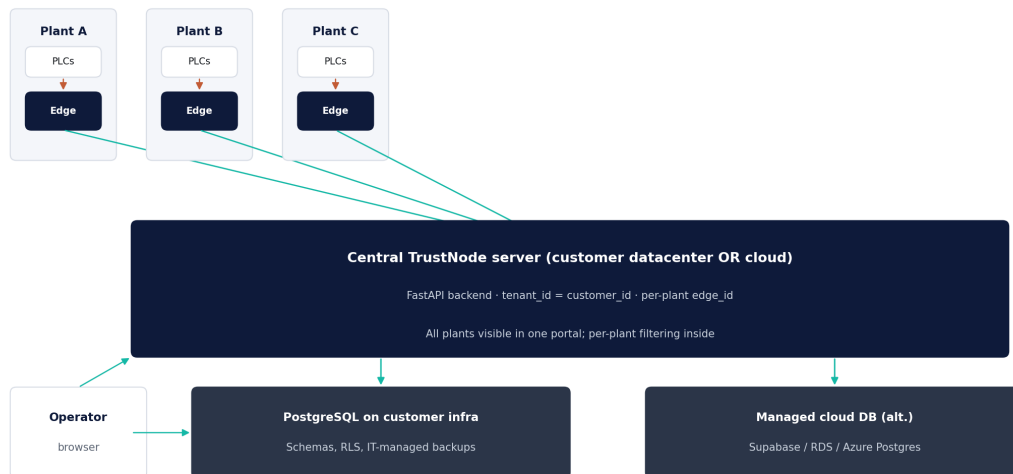


TrustNode Edge — Security & Architecture

Figure D — Cloud-bridged (the reference topology).

Deployment E — Multi-plant central historian

One customer, several plants, a central database that aggregates them all.



TrustNode Edge — Security & Architecture

Figure E — Multi-plant central historian.

14. Storage options — where the data physically lives

The historian is portable; you choose where it lives. Five options today, summarised in **Figure F** (storage_options.png).

Why does this matter? Industrial data is your data. Some customers have strict policies — data must stay in-country, on-prem, or in a specific provider. We support all the common cases without forcing a single shape on you.

1. Local SQLite only (edge-only)

- **What it is:** a single file on disk at the edge (trustnode_app_store.db). The simplest possible deployment.
- **Pros:** zero network DB, smallest attack surface, fastest install, backup = copy the file.
- **Cons:** one box; if that box dies, the historian dies with it (until you restore the file).
- **Best for:** small standalone sites, demo / pilot installs, air-gapped environments.

2. Customer-owned PostgreSQL

- **What it is:** PostgreSQL running on a server the customer owns — either next to the edge or in their datacenter.
- **Pros:** customer's existing BI tools (Power BI, Tableau, Grafana) can query it directly, customer's existing backup policy applies, full control by customer IT.

- **Cons:** customer must operate Postgres (upgrades, backups, monitoring). Most large customers already do this.
- **Best for:** mid/large customers with DBAs, regulated industries where data residency matters.

3. Managed cloud database

Three concrete options today (others on request):

Provider	Notes
Supabase	Managed Postgres, includes auth and storage; we use it for our managed cloud by default. EU and US regions.
AWS RDS for PostgreSQL	Industry-standard managed Postgres on AWS. Customer's AWS account, our deployment.
Azure Database for PostgreSQL	Microsoft's managed Postgres. Common for customers already on Azure.

- **Pros:** provider handles backups, encryption-at-rest, high availability, point-in-time restore. Pay-as-you-grow.
- **Cons:** monthly cost (typically EUR 25–250/month depending on size), customer's data leaves their datacenter.
- **Best for:** customers without DBAs, multi-region / multi-plant scenarios, customers who want fast time-to-value.

4. Hybrid (edge SQLite + cloud DB mirror)

- **What it is:** the edge keeps a full local copy in SQLite AND syncs to a cloud Postgres in the background.
- **Pros:** best resilience — cloud outage = zero data loss because the local copy is the source of truth. Plant continues even if WAN goes down for hours.
- **Cons:** slightly more disk space at the edge, slightly more bandwidth than cloud-only.
- **Best for:** production-critical sites, expensive downtime, regulated reporting where every minute must be captured.

5. Other databases on request

We can adapt to other engines if the customer mandates one:

- **MySQL / MariaDB** — straightforward, similar SQL dialect to Postgres.
- **Microsoft SQL Server** — common in Windows-only enterprises.
- **TimescaleDB** — Postgres extension optimised for time-series data; good fit for very high-cardinality historians.

- **InfluxDB / QuestDB** — pure time-series databases; we can write the historian into them while keeping config in Postgres or SQLite.

These require a small schema adapter; tell us your stack and we will scope it.

Database comparison summary

Option	Local	On-prem DB	Cloud DB	Hybrid	Best for
1. SQLite only	✓	—	—	—	Smallest sites, air-gap
2. Customer Postgres	optional	✓	—	—	Customer with DBAs
3. Managed cloud	optional	—	✓	—	Fast time-to-value, multi-plant
4. Hybrid	✓	optional	✓	✓	Production-critical, zero loss
5. Other DB on request	—	✓	✓	varies	Customer-mandated stack

Storage options

Five ways to hold your data — pick what fits IT policy and budget.

1. Local SQLite only	2. Customer PostgreSQL	3. Managed cloud DB	4. Hybrid edge + cloud	5. Other DBs on request
Edge-only, single file on disk.	Customer hosts the DB themselves.	Supabase, AWS RDS, Azure Postgres.	Edge SQLite + cloud DB mirror.	MySQL, MSSQL, TimescaleDB.
• No cloud, no network DB • Lightest install, smallest attack surface • Backup = copy the SQLite file • Best for: small standalone sites	• Full control by customer IT • Customer's existing BI tools query it • Customer's existing backup policy applies • Best for: mid/large customer with DBAs	• Provider handles backups + HA • Encryption-at-rest standard • Pay-as-you-grow scaling • Best for: multi-plant, multi-region	• Local-only fallback • Cloud-level redundancy • Cloud outage = zero data loss • Best for: production-critical sites	• Supported on request • May require schema adapter • Air-gapped variants possible • Best for: customer-mandated stacks

TrustNode Edge — Security & Architecture

Figure F — Five storage options at a glance.

15. Networking prerequisites — what TrustNode needs from your network

Every deployment shares the same fundamental network needs. This section is the checklist your IT and OT teams can run through together.

15.1 What TrustNode must reach

- **PLC IP reachability.** The TrustNode host (PC, IPC, server, or VM) must be able to reach every PLC it polls on the configured IP and TCP port. Typical ports:

Protocol	Default port	Direction
Modbus TCP	502/TCP	TrustNode → PLC
OPC UA	4840/TCP (and any custom)	TrustNode → server
Siemens S7 (Snap7)	102/TCP (ISO-on-TCP)	TrustNode → PLC
Allen-Bradley EtherNet/IP	44818/TCP and 2222/UDP	TrustNode → PLC

- **DNS resolution.** TrustNode is fine with IPs or hostnames. If you use hostnames for PLCs, the host must resolve them.
- **Time sync (NTP).** Historian data is only useful if timestamps are correct. We recommend NTP from the customer's own time source (domain controller or an OT-side NTP server).

15.2 What TrustNode does NOT need

- **Inbound ports on the edge.** None. The edge does not accept inbound connections from outside the host except the local loopback for the Electron desktop UI.
- **Direct internet access from the PLCs.** PLCs stay on their VLAN; only TrustNode talks to them.
- **An open firewall to the cloud.** Outbound TCP/443 only — same direction as a web browser fetching a page.

15.3 Special hardware (when needed)

- **NAT / firewall.** Almost every customer already has one. We need exactly one outbound rule: HTTPS/443 from the TrustNode host to the cloud VPS. No inbound rule.
- **VLAN segmentation.** Recommended — keep PLCs on their own VLAN, and put TrustNode on a host that has one foot in the OT VLAN and one foot in the IT VLAN (typical "data-bridge" pattern).
- **Cellular / VPN router** (optional, common with IPC deployments). A 4G/5G modem or LTE router fitted in the cabinet gives the IPC outbound internet without needing the customer's corporate WAN.
- **Proxy server.** If the customer enforces outbound HTTP proxy, we support HTTPS_PROXY / HTTP_PROXY environment variables.

- **Custom TLS root CA.** If the customer terminates TLS at an enterprise proxy with their own CA, we mount the CA bundle into the edge and the cloud connection works through it.

15.4 Per-topology firewall summary

Topology	Customer firewall needs	Internet needed?
A — Plant PC	Allow LAN ↔ PLC VLAN	No
B — IPC in panel	Allow IPC ↔ PLC backplane	No (unless cloud portal fitted)
C — Customer server	Allow server ↔ each plant's PLCs (often over the customer's WAN/VPN)	No
D — Cloud-bridged	Outbound HTTPS from edge to cloud (TCP/443)	Yes
E — Multi-plant central	Outbound HTTPS from each plant edge to central server	Yes (or customer WAN)

15.5 Air-gap and "no-cloud" customers

Some industrial customers — defence, utilities, certain pharmaceuticals — cannot put any data in any cloud. TrustNode supports this:

- Run topology **A** or **B**, all data in local SQLite.
- No cloud subdomain, no portal, no outbound HTTPS — disable the cloud sync at install.
- Operators connect to the edge over the LAN. Reports and dashboards run locally.
- Software updates are delivered as offline installers handed to the customer's change-management process.

16. Backup and redundancy — what's on us, what's on you

Who runs the backups depends on which storage option you picked.

Storage option	Who handles backups	Redundancy options
1. SQLite only	Customer (copy the file on a schedule)	Periodic file copy to a fileshare / NAS / external disk
2. Customer Postgres	Customer IT	Postgres replication, base backups, customer's standard DB ops
3. Managed cloud DB	Provider	Provider's built-in daily backups, point-in-time restore, multi-AZ replicas
4. Hybrid	Both	Cloud-side backups + local SQLite as a permanent backup-of-record
5. Other DB on request	Customer or provider	Whatever that DB engine supports

What TrustNode itself adds on top:

- The edge always retains a recent local buffer of historian rows in SQLite, regardless of which long-term storage you picked. This means even a brief loss of the long-term DB cannot lose data already polled.
- Configuration (gateways, tags, dashboards, reports) is versioned in the cloud Supabase mirror when you use topology D or E. Versions are queryable.
- Activation, license, and audit data live in the cloud control plane — backed up by Supabase / your DB provider.

High availability options on request:

- Active-passive edge pair (two edges on the same PLC network, one warm standby).
- Cloud backend behind a load balancer with two FastAPI instances.
- Managed Postgres in HA mode (Supabase paid tier, RDS Multi-AZ, Azure Zone-redundant).

We deliberately don't make these defaults — most customers don't need them, and the cost is meaningful. If you do, ask and we'll scope.

17. Architecture diagrams

See the rendered PNG diagrams in the `diagrams/` folder. They cover:

Core architecture:

- `architecture_single_customer.png` — full single-customer deployment.
- `architecture_purdue.png` — Purdue Model with TrustNode mapped onto it.
- `architecture_three_role.png` — what each login role can see and change.

- `architecture_multi_tenant.png` — how multiple customer subdomains stay isolated on one VPS.
- `architecture_store_forward.png` — how the edge keeps recording during a cloud outage.

Deployment topologies (§13):

- `deployment_plant_pc.png` — A — Plant PC / desktop install.
- `deployment_ipc_panel.png` — B — IPC in electrical panel.
- `deployment_customer_server.png` — C — Customer server in datacenter.
- `deployment_cloud_bridged.png` — D — Cloud-bridged (the default).
- `deployment_multi_plant.png` — E — Multi-plant central historian.

Storage options (§14):

- `storage_options.png` — five storage options side by side.

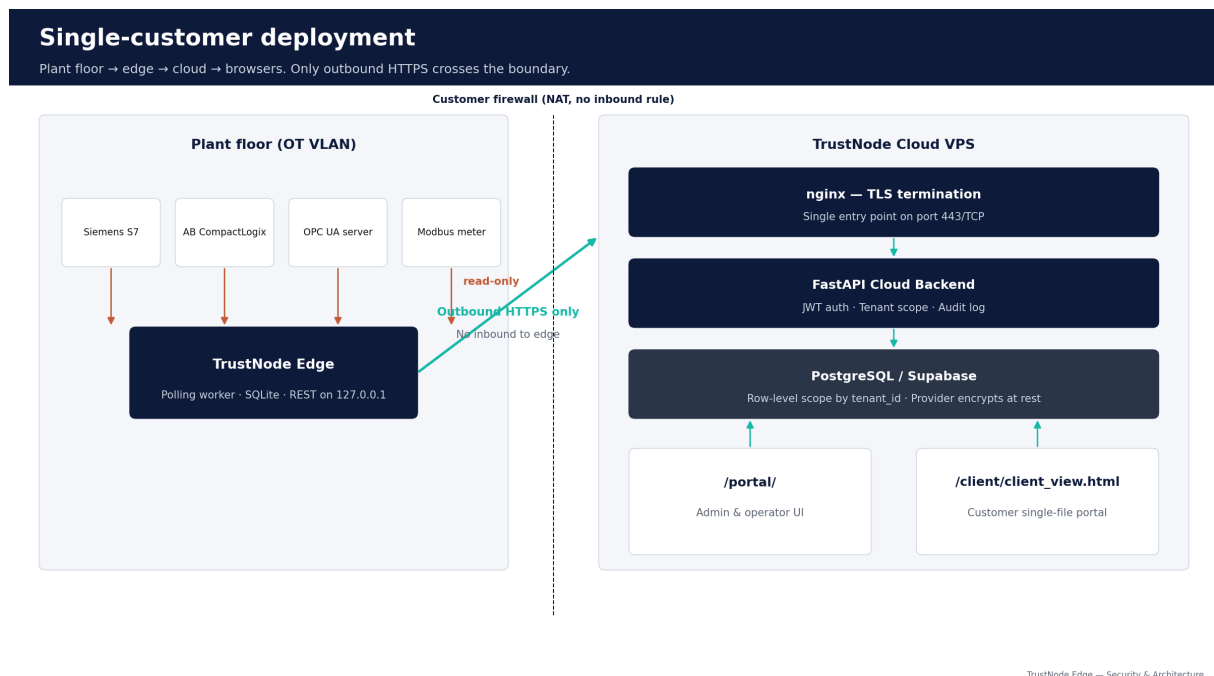


Figure G — Full single-customer deployment overview.

18. The minute-by-minute version (for the pitch meeting)

If you have 10 minutes with a prospect's CISO, this is the order:

1. **30 seconds — what we are:** TrustNode is a read-only industrial data gateway. PLC → database → dashboard.
2. **60 seconds — show the architecture diagram (single-customer).** Point at the OT/IT boundary. "We are outbound-only across this line."

3. **60 seconds — three guarantees:** read-only on PLCs, outbound-only on the network, tenants are isolated.
 4. **2 minutes — three layers of tenant isolation:** show the tenant-scope diagram. Mention RLS as an additional backstop.
 5. **2 minutes — compliance:** point at the IEC 62443 / NIS2 / ISO 27001 mapping. We do not claim certification; we provide the evidence package.
 6. **2 minutes — what we ask of the customer:** the five-item checklist in §12.
 7. **2 minutes — Q&A.** Most questions will be answered by §19 below.
-

19. FAQ for prospect calls

Q: Can TrustNode write to my PLCs? No. There is no write code path in the gateway. We only read.

Q: Do I need to open an inbound firewall port for the edge? No. The edge dials out to the cloud over HTTPS/443, exactly like a web browser.

Q: Can another customer on the same cloud see my data? No. Every database query, every API call, and every live stream is filtered by `tenant_id` before it leaves the server. Three independent automated smokes verify this on every release.

Q: Where does my data physically live? You choose. Options: edge-only (SQLite), your own PostgreSQL, our managed Supabase, or a hybrid. The historian schema is portable; you can migrate at any time.

Q: What happens if the cloud goes down? The edge keeps polling PLCs and stores readings locally. When the cloud is back, it catches up incrementally. The plant floor sees no interruption.

Q: How are passwords stored? PBKDF2-HMAC-SHA256 with 120,000 iterations and per-user salt. We never store plaintext, never log it, and never email it.

Q: How do you stop someone with a leaked JWT from logging in forever? Tokens expire in 12 hours. There is no refresh token; the user re-authenticates. For high-sensitivity deployments, this can be lowered.

Q: Is this IEC 62443 certified? Not certified yet. The architecture follows IEC 62443 zone-and-conduit principles, IEC 62443-3-3 system security requirements, and the Purdue Reference Model. We provide this document and the smoke-test scripts as evidence for your compliance team.

Q: What about NIS2? NIS2 is a regulatory framework; certification is not the right concept. Our practices (segregation, audit log, supply-chain pinning, incident-ready logging, business-continuity via store-and-forward) align with NIS2 Article 21(2) risk-management measures. We're happy to walk through each one with your DPO/CISO.

Q: Can you give me an architecture review session with our security team? Yes — this document is the starting point. We can run a live walkthrough of the code paths it cites.

Q: Do you support our existing identity provider (Azure AD / Okta / Keycloak)? SAML SSO is a deployment option, not the default. Talk to us about your IdP and we will scope it.

20. Glossary — every acronym, in plain English

Term	Stands for	One-line explanation
API	Application Programming Interface	The set of network endpoints (URLs) the software exposes for other programs to call.
AES	Advanced Encryption Standard	The standard symmetric encryption algorithm used in most modern systems.
BitLocker	—	Microsoft's full-disk encryption built into Windows.
CISO	Chief Information Security Officer	The executive responsible for an organisation's cybersecurity posture.
CIP	Common Industrial Protocol	The protocol family used by Rockwell/Allen-Bradley PLCs.
DCS	Distributed Control System	An older-style industrial control system, common in process industries (oil/gas, chemicals).
DMZ	Demilitarised Zone	A buffer network between two more sensitive networks.
EtherNet/IP	—	The Ethernet-based variant of CIP, used by Rockwell/Allen-Bradley.
FIPS	Federal Information Processing Standards	US government cryptographic standards (FIPS 140-2/3 is the relevant one for TLS).
HMAC	Hash-based Message Authentication Code	A way to use a hash function together with a secret key to prove a message wasn't altered.
HTTPS	HyperText Transfer Protocol Secure	HTTP carried over TLS — the encrypted form of normal web traffic.
HMI	Human-Machine Interface	The screen an operator looks at to monitor and control a process.

Term	Stands for	One-line explanation
IEC	International Electrotechnical Commission	The international body that publishes standards for electrical/electronic systems, including IEC 62443.
IT	Information Technology	Enterprise/office computing — emails, file servers, business applications.
JWT	JSON Web Token	An internet-standard format for a signed authentication token.
LUKS	Linux Unified Key Setup	The Linux full-disk encryption standard.
mTLS	Mutual TLS	A TLS connection where both sides (not just the server) prove identity with a certificate.
Modbus	—	A simple, register-based industrial protocol; near-universal among field devices.
NAT	Network Address Translation	The router trick that lets many devices share one public internet address.
NIS2	Network and Information Security 2 (EU Directive)	The EU's cybersecurity directive for critical-infrastructure operators.
OPC UA	Open Platform Communications, Unified Architecture	The modern, vendor-neutral industrial protocol; standard IEC 62541.
OT	Operational Technology	The plant-floor computing world — PLCs, DCSS, SCADA.
PBKDF2	Password-Based Key Derivation Function 2	A password-stretching algorithm — slows down brute-force attacks.

Term	Stands for	One-line explanation
PLC	Programmable Logic Controller	The industrial computer that actually controls a machine or process.
REST	Representational State Transfer	The web-API style we use — HTTP verbs (GET/POST/PUT/DELETE) on URL endpoints.
RLS	Row-Level Security	A database-enforced rule that filters which rows a query can return per user.
RDP	Remote Desktop Protocol	Microsoft's protocol for remote graphical desktop access. (We do not use it.)
SAML	Security Assertion Markup Language	A common single-sign-on protocol used in corporate environments.
SCADA	Supervisory Control And Data Acquisition	The older umbrella name for an industrial monitoring/control system.
SHA-256	Secure Hash Algorithm — 256 bits	A one-way cryptographic hash function widely used (HTTPS, JWT, blockchain).
SSE	Server-Sent Events	A one-way HTTP push protocol — server streams short updates to the browser.
SSO	Single Sign-On	Sign in once with your corporate identity, gain access to many apps.
SQL	Structured Query Language	The language used to talk to relational databases like PostgreSQL.
TCP	Transmission Control Protocol	The reliable, connection-oriented network protocol HTTP and TLS run on.
TLS	Transport Layer Security	The modern name for SSL — the encryption used by HTTPS.

Term	Stands for	One-line explanation
TTL	Time To Live	How long a token or cache entry is valid before it expires.
VLAN	Virtual Local Area Network	A way to logically separate devices on the same physical switch.
VFD	Variable-Frequency Drive	An industrial device that controls motor speed; common Modbus client.
VPS	Virtual Private Server	A rented Linux server in a data centre. The TrustNode cloud runs on one.
WebSocket	—	A protocol for two-way persistent connections, normally over HTTPS. We use it locally only.
WSS	WebSocket Secure	The encrypted version of WebSocket, running over TLS.

Document maintained by the TrustNode engineering team. Last updated 2026-05-15.

Customer-facing — share freely.